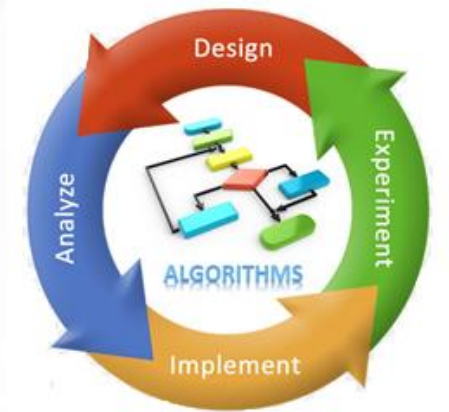


Unit-4

Dynamic Programming



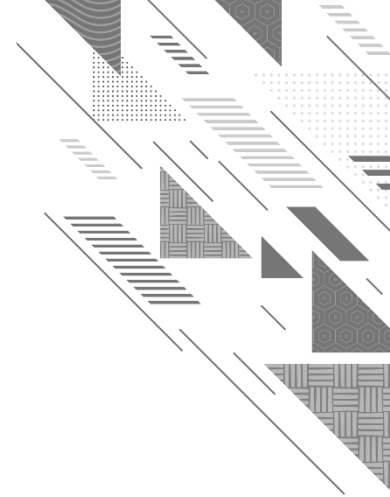
SUBSCRIBE





Outline

- Introduction
- The Principle of optimality
- Problem Solving using Dynamic Programming – Calculating the Binomial Coefficient
- Making Change Problem
- Knapsack Problem
- Assembly Line-Scheduling
- All Points Shortest Path
- Matrix Chain Multiplication
- Longest Common Subsequence



Introduction



Introduction



- A box contains 4 stones worth \$1400, \$3000, \$4200 and \$7100. They weigh 20, 50, 60 and 100 grams respectively.
- Your bag can only hold **110 grams**.

- Greedy Approach

v_i	1400	3000	4200	7100
w_i	20	50	60	100
v_i/w_i	70	60	70	71

- The greedy algorithm selects the best candidate with the highest profit, i.e., **stone worth \$7100**.

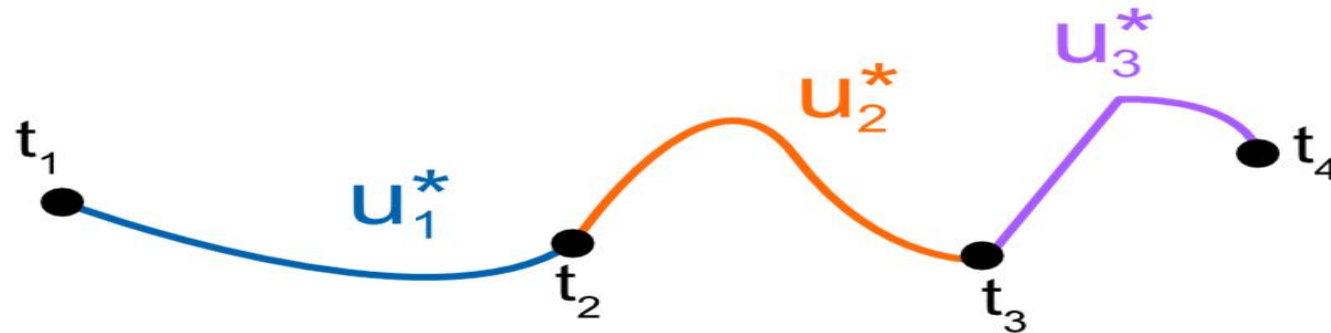
Introduction



- A box contains 4 stones worth \$1400, \$3000, \$4200 and \$7100. They weigh 20, 50, 60 and 100 grams respectively.
- Your bag can only hold **110 grams**.
- But, the optimal choice would be to select the two stones worth **\$3000 and \$4200**.
- Their combined weight is **110 grams**, which means they will fit into your bag and you will get a profit of **\$7200**.
- If there are 100 stones, how to efficiently find the optimal answer?

The Principle of Optimality

- ▶ A dynamic-programming algorithm solves every sub-problem just once and then **saves its answer in a table**.
- ▶ It avoids the work of **re-computing the answer** every time the sub problem is encountered.
- ▶ The dynamic programming algorithm obtains the solution using **principle of optimality**.
- ▶ The principle of optimality states that “in an optimal sequence of decisions or choices, each subsequence must also be optimal.”
- ▶ If it is **not possible to apply the principle of optimality** then it is almost **impossible to obtain the solution** using the dynamic programming approach.





Binomial Coefficient



Introduction

- ▶ If you want to make a 2-person committee from a group of four people. How many **different combinations** are possible?
- ▶ The number of ways to do this is given by $C(4, 2)$.

Consider 4 people A, B, C, D. Now the different combinations for making a committee of 2 persons from these 4 are;

A, B

A, C

A, D

C, D

B, D

B, C

$$C(n, k) = \binom{n}{k} = \frac{n!}{k!(n-k)!}$$

$$C(4, 2) = \binom{4}{2} = \frac{4!}{2!(4-2)!} = 6$$

- ▶ Specifically, the **binomial coefficient** $C(n, k)$ counts the number of ways to form **an unordered collection of k items** chosen from a collection of **n** distinct items.

Binomial Coefficient

- The definition of binomial coefficient is given as:

$$\binom{n}{k} = \begin{cases} 1 & \text{if } k = 0 \text{ or } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{if } 0 < k < n \\ 0 & \text{Otherwise} \end{cases}$$

```
function C(n, k)
    if k=0 or k=n then return 1
    else return C(n-1, k-1) + C(n-1, k)
```

Binomial Coefficient

	0	1	2	3	4	5	..	k
0								
1		→ ⊕ ←						
2		→ ⊕ ←	→ ⊕ ←					
3		→ ⊕ ←	→ ⊕ ←	→ ⊕ ←				
4								
..								
n								

PASCAL's
TRIANGLE

```
function C(n, k)
  if k=0 or k=n then return 1
  else return C(n-1, k-1) + C(n-1, k)
```

Generalized Solution using Dynamic Programming

1. Characterize the **structure** of an optimal solution.
2. Recursively **define the value** of an optimal solution.
3. Compute the value of an optimal solution in a **bottom-up fashion**.
4. Construct **an optimal solution** from the computed information.



Make a Change Problem



Make Change Problem – Dynamic Programming Solution

- We need to generate a table $c[n][N]$, where
1. n = number of denominations
 2. N = amount for which you need to make a change.

To generate table $c[i][j]$ use following steps:

Step-1: Make $c[i][0] = 0$ for $0 < i \leq n$

Repeat step-2 to step-4 for the remaining matrix values

Step-2: If $i = 1$ then $c[i][j] = 1 + c[1][j - d_1]$

Step-3: If $j < d_i$ then $c[i][j] = c[i - 1][j]$

Step-4: Otherwise $c[i][j] = \min(c[i - 1][j], 1 + c[i][j - d_i])$

Optimal
Sub-structure

Make Change Problem – Dynamic Programming Solution

► Denominations: $d_1 = 1, d_2 = 4, d_3 = 6$. Make a change of **Rs. 8.**

Step-1: Make $c[i][0] = 0$ for $0 < i \leq n$

$j \longrightarrow$ Amount

$i = 1$									
$i = 2$									
$i = 3$									

$$\min(c[1][5], 1 + c[2][0]) = \min(5, 1 + 0) = \min(5, 1) = 1$$

Make Change Problem – Dynamic Programming Solution

► Denominations: $d_1 = 1, d_2 = 4, d_3 = 6$. Make a change of **Rs. 8.**

Step-1: Make $c[i][0] = 0$ for $0 < i \leq n$

Step-2: If $i = 1$ then $c[i][j] = 1 + c[1][j - d_1]$, here $d_1 = 1$

Step-3: If $j < d_i$ then $c[i][j] = c[i - 1][j]$

Step-4: Otherwise $c[i][j] = \min(c[i - 1][j], 1 + c[i][j - d_i])$

$j \longrightarrow$ Amount

		0	1	2	3	4	5	6	7	8
$i = 1$	$d_1 = 1$	0	1	2	3	4	5	6	7	8
$i = 2$	$d_2 = 4$	0	1	2	3	1	2			
$i = 3$	$d_3 = 6$	0								

Make Change Problem – Dynamic Programming Solution

► We can also find the coins to be included in the solution set as follows:

1. Start looking at $c[3, 8] = c[2, 8] \Rightarrow$ So, **not to include** a coin with denomination 6.

2. Next go to $c[2, 8] \neq c[1, 8]$ but $c[2, 8] = 1 + c[2, 4]$

$$c[i][j] = \min(c[i-1][j], 1 + c[i][j - d_i])$$

▪ So, include a coin with denomination 4

3. Now, got to $c[2, 4] \neq c[1, 4]$ but $c[2, 4] = 1 + c[2, 0]$

▪ So, again include a coin with denomination 4

4. Go to $c[2, 0] = c[1, 0]$ and stop.

	0	1	2	3	4	5	6	7	8
$d_1 = 1$	0	1	2	3	4	5	6	7	8
$d_2 = 4$	0	1	2	3	①	2	3	4	②
$d_3 = 6$	0	1	2	3	1	2	1	2	2

Solution contains 2 coins with denomination 4



0/1 Knapsack Problem



0/1 Knapsack Problem - Dynamic Programming Solution

► We need to generate table $V(1 \dots n, 0 \dots W)$

1. where, n = number of objects.
2. W = capacity of knapsack.

To generate table $V[i][j]$ use following steps:

Step-1: Make $V[i][0] = 0$ for $0 < i \leq n$

Step-2: if $j < w_i$ then

$$V[i][j] = V[i - 1][j]$$

Step-3: if $j \geq w_i$ then

$$V[i][j] = \max(V[i - 1][j], V[i - 1][j - w_i] + v_i)$$

0/1 Knapsack Problem - Dynamic Programming Solution

► Solve the following knapsack problem using dynamic programming technique.

1. $W = 11$ and $n = 5$

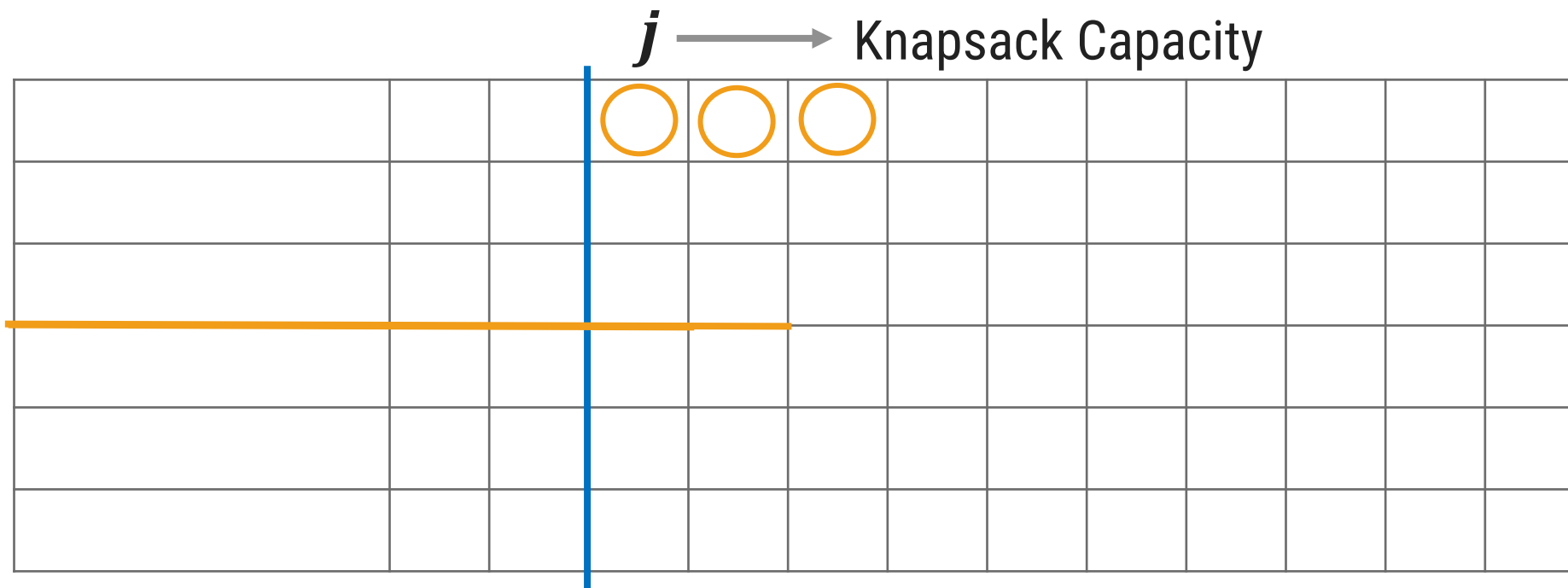
Object i	1	2	3	4	5
v_i	1	6	18	22	28
w_i	1	2	5	6	7

0/1 Knapsack Problem - Dynamic Programming Solution

$V[i][0] = 0$ for $0 < i \leq n$

if $j < w_i$ then $V[i][j] = V[i - 1][j]$

else $V[i][j] = \max(V[i - 1][j], V[i - 1][j - w_i] + v_i)$



0/1 Knapsack Problem - Dynamic Programming Solution

$V[i][0] = 0$ for $0 < i \leq n$

if $j < w_i$ then $V[i][j] = V[i-1][j]$

else $V[i][j] = \max(V[i-1][j], V[i-1][j - w_i] + v_i)$

$V[3][6]$, here $i = 3, j = 6, w_3 = 5, v_3 = 18$

So, $j \geq w_3$

$$V[3][6] = \max(V[2][6], V[2][1] + v_3) \\ = \max(7, 1 + 18) = 19$$

$j \longrightarrow$ Knapsack Capacity

weight & value	0	1	2	3	4	5	6	7	8	9	10	11
$w_1 = 1, v_1 = 1$	0	1	1	1	1							
$w_2 = 2, v_2 = 6$	0	1	6	7	7							
$w_3 = 5, v_3 = 18$	0	1	6	7	7							
$w_4 = 6, v_4 = 22$	0	1	6	7	7							
$w_5 = 7, v_5 = 28$	0	1	6	7	7							

0/1 Knapsack Problem - Dynamic Programming Solution

► We can also find the objects to be carried in the knapsack as follows,

1. Start looking at $V[5, 11] = V[4, 11] \Rightarrow$ So **not to include** object 5
2. Next go to $V[4, 11] \neq V[3, 11]$ but $V[4, 11] = V[3, 11 - w_4] + v_4 = V[3, 5] + 22$ So **include object 4**
3. Now go to $V[3, 5] \neq V[2, 5]$ but $V[3, 5] = V[2, 5 - w_3] + v_3 = V[2, 0] + 18$ So **include object 3**
4. Now $V[2, 0] = V[1, 0]$ and $V[1, 0] = V[0, 0] \Rightarrow$ So objects 2 and 1 are **not included**.

$$V[i][j] = \max(V[i-1][j], V[i-1][j - w_i] + v_i)$$

weight & value	0	1	2	3	4	5	6	7	8	9	10	11
$w_1 = 1, v_1 = 1$	0	1	1	1	1	1	1	1	1	1	1	1
$w_2 = 2, v_2 = 6$	0	1	6	7	7	7	7	7	7	7	7	7
$w_3 = 5, v_3 = 18$	0	1	6	7	7	18	19	24	25	25	25	25
$w_4 = 6, v_4 = 22$	0	1	6	7	7	18	22	24	28	29	29	40
$w_5 = 7, v_5 = 28$	0	1	6	7	7	18	22	28	29	34	35	40

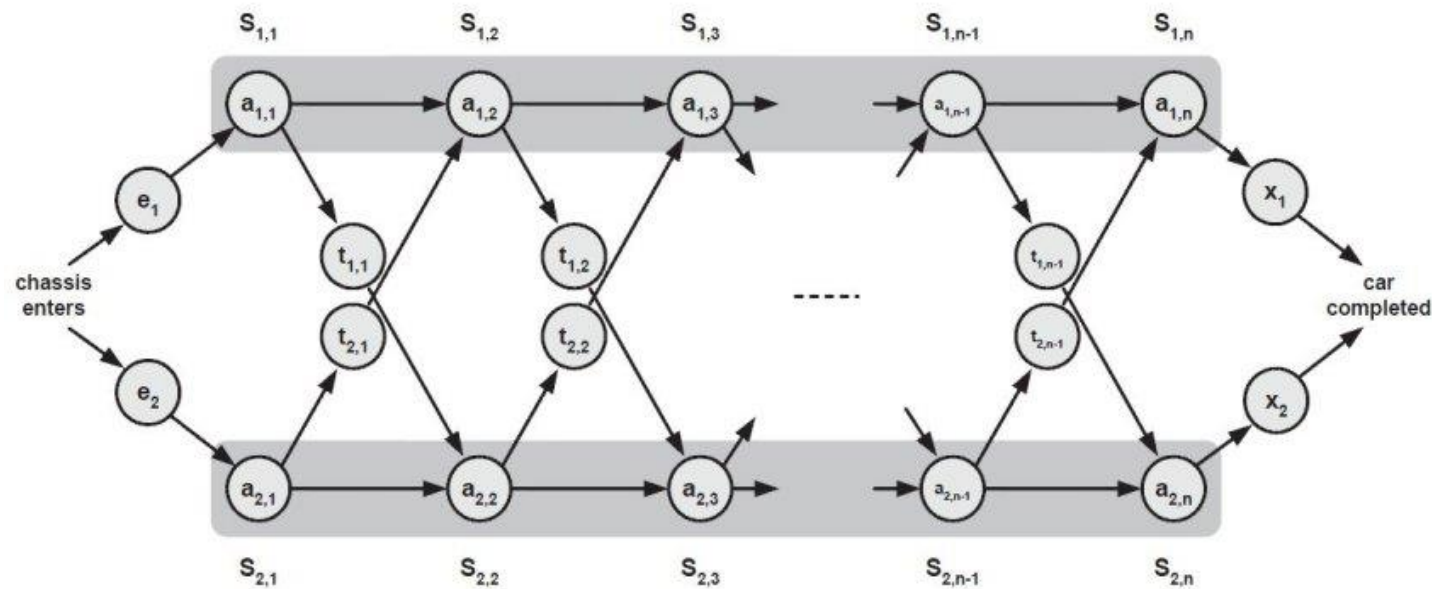


Assembly Line Scheduling



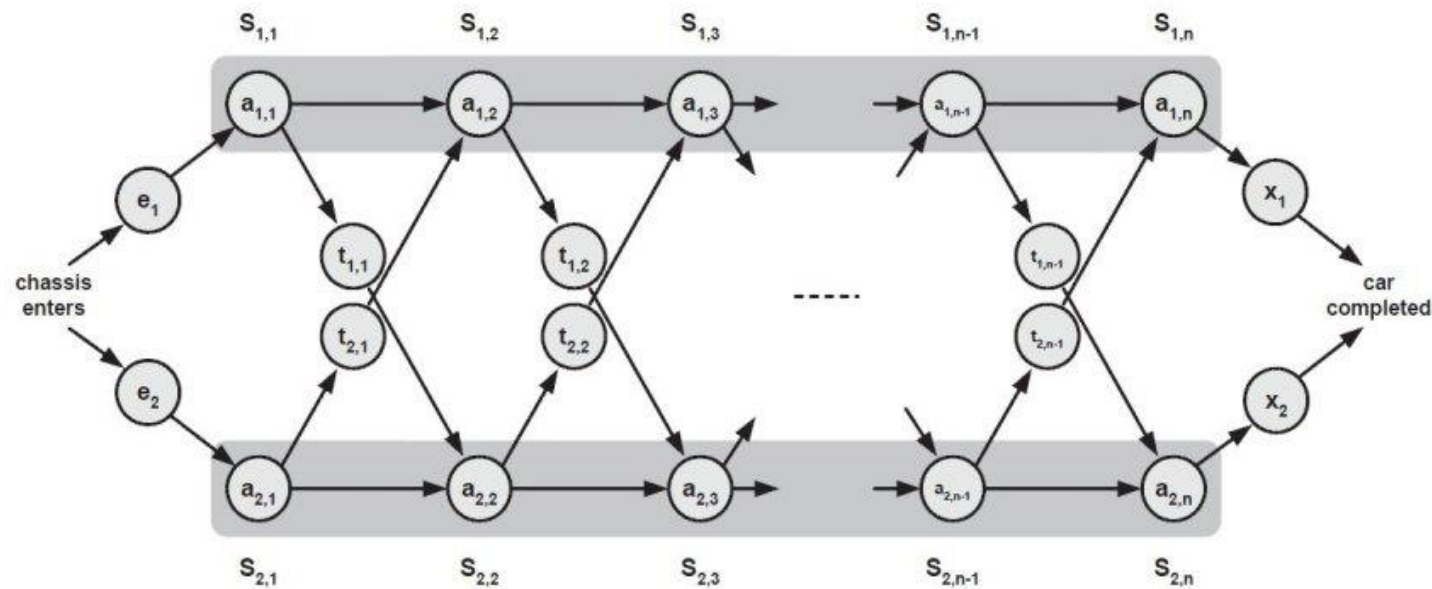
Introduction

- ▶ An automobile chassis enters each assembly line, has parts added to it at a number of **stations**, and a finished auto exits at the end of the line.
- ▶ There are two assembly lines, numbered $i = 1, 2$.
- ▶ Each assembly line has n stations, numbered $j = 1, 2, \dots, n$



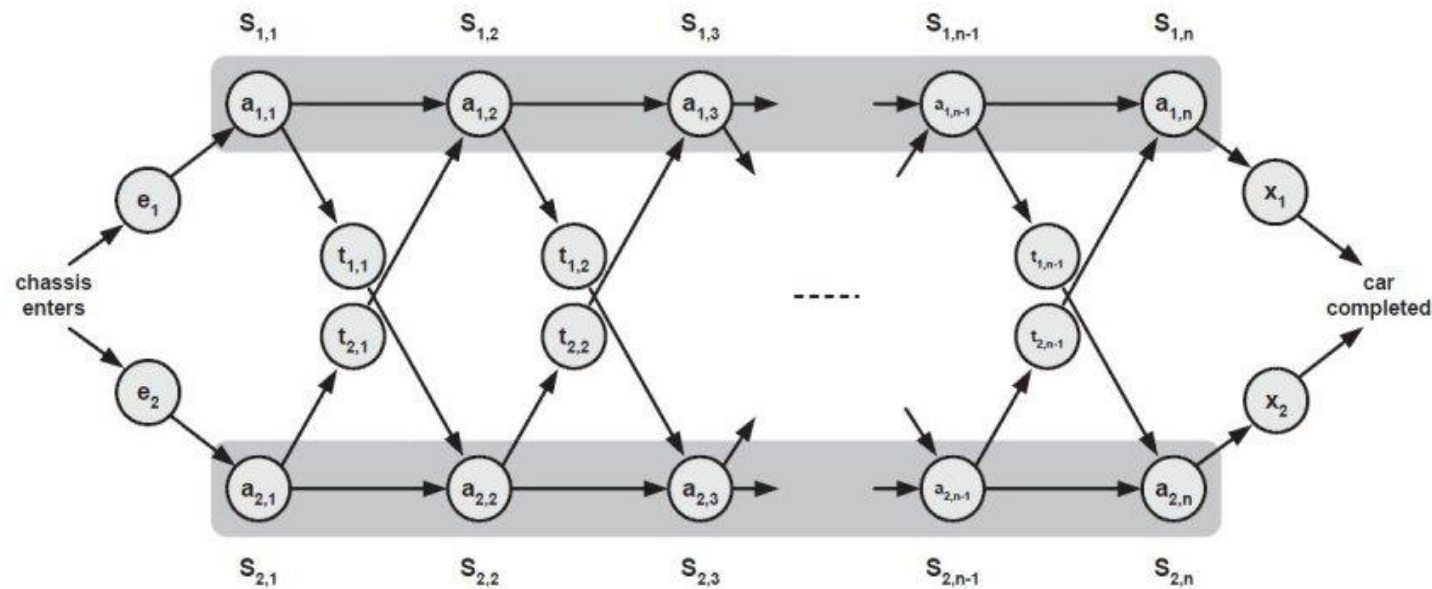
Introduction

- ▶ We denote the j^{th} station on line i by $S_{i,j}$.
- ▶ The j^{th} station on line 1 ($S_{1,j}$) performs the same function as the j^{th} station on line 2 ($S_{2,j}$).
- ▶ Corresponding stations $S_{1,j}$ and $S_{2,j}$ perform the same function **but can take different amounts of time** $a_{1,j}$ and $a_{2,j}$.



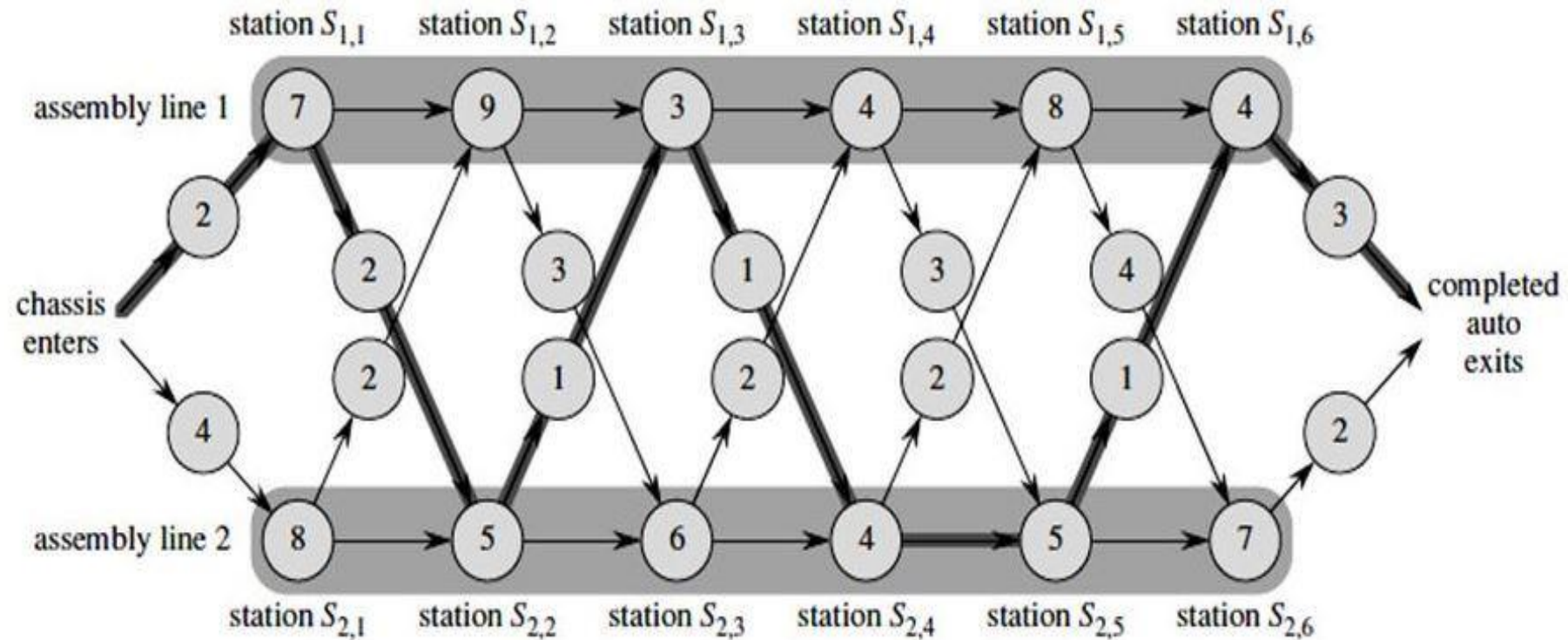
Introduction

- ▶ Entry times are: e_1 and e_2 ; exit times are: x_1 and x_2
- ▶ After going through a station, a chassis can either:
 - ➔ stay on **same line** at no cost, or
 - ➔ transfer to **other line**: cost after $s_{i,j}$ is $T_{i,j}$ for $j = 1, \dots, n-1$



Assembly Line Scheduling - Example

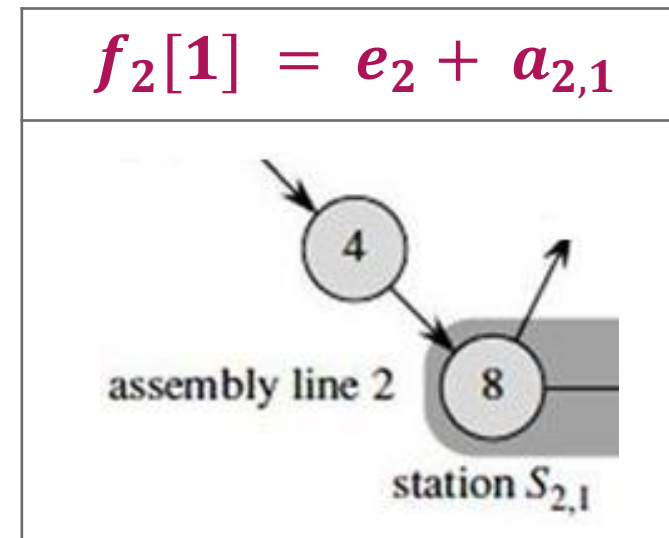
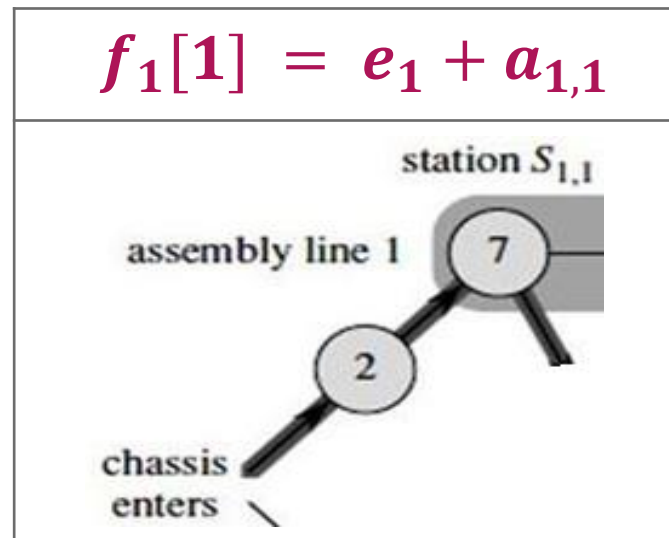
- ▶ Using dynamic programming technique, find the fastest time to get through the entire factory for the following:



Optimal Substructure

Step 1:

- ▶ f^* : the fastest time to get through the entire factory
- ▶ $f_i[j]$: the fastest time to get from the starting point through station $S_{i,j}$.
- ▶ Base case: $j = 1, i = 1, 2$ (getting through station 1)



Assembly Line Scheduling – Dynamic Programming Solution

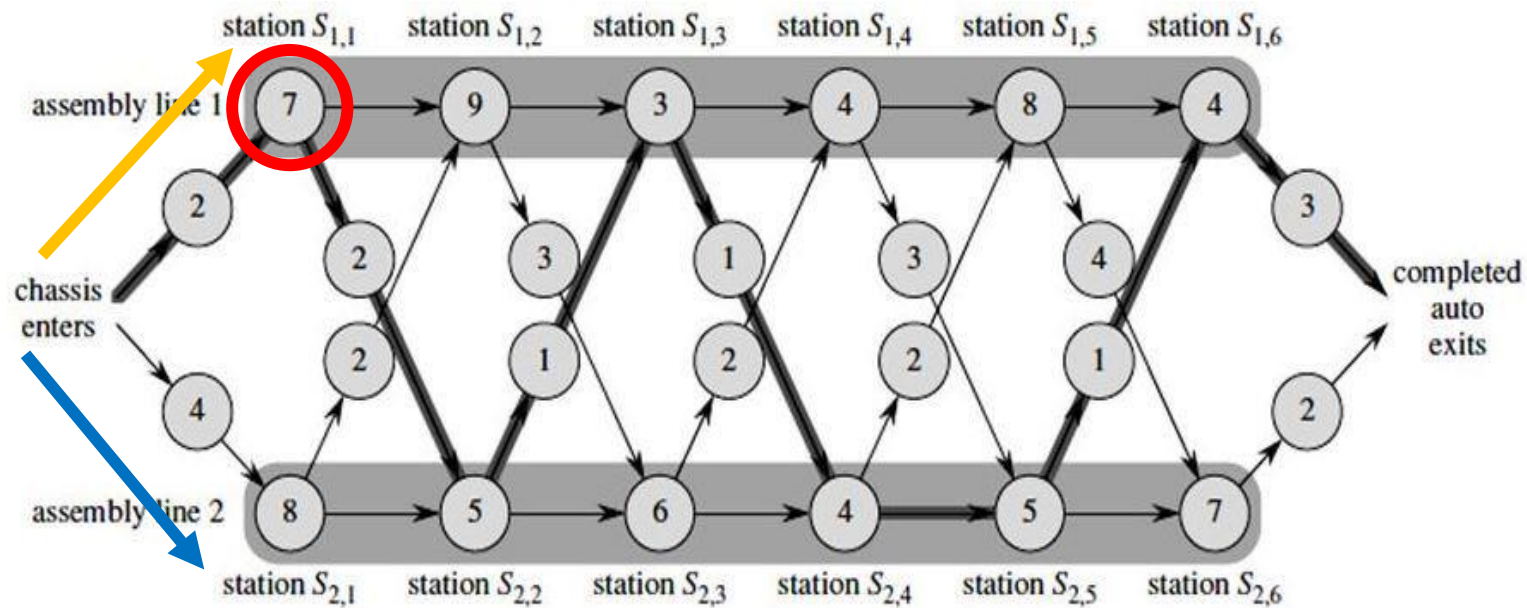
- ▶ General Case: $j = 2, 3, \dots, n$ and $i = 1, 2$ - Recursive solution

$$f_1[j] = \begin{cases} e_1 + a_{1,1} & \text{if } j = 1, \\ \min(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j}) & \text{if } j \geq 2 \end{cases}$$

$$f_2[j] = \begin{cases} e_2 + a_{2,1} & \text{if } j = 1, \\ \min(f_2[j-1] + a_{2,j}, f_1[j-1] + t_{1,j-1} + a_{2,j}) & \text{if } j \geq 2 \end{cases}$$

$$f^* = \min(f_1[n] + x_1, f_2[n] + x_2)$$

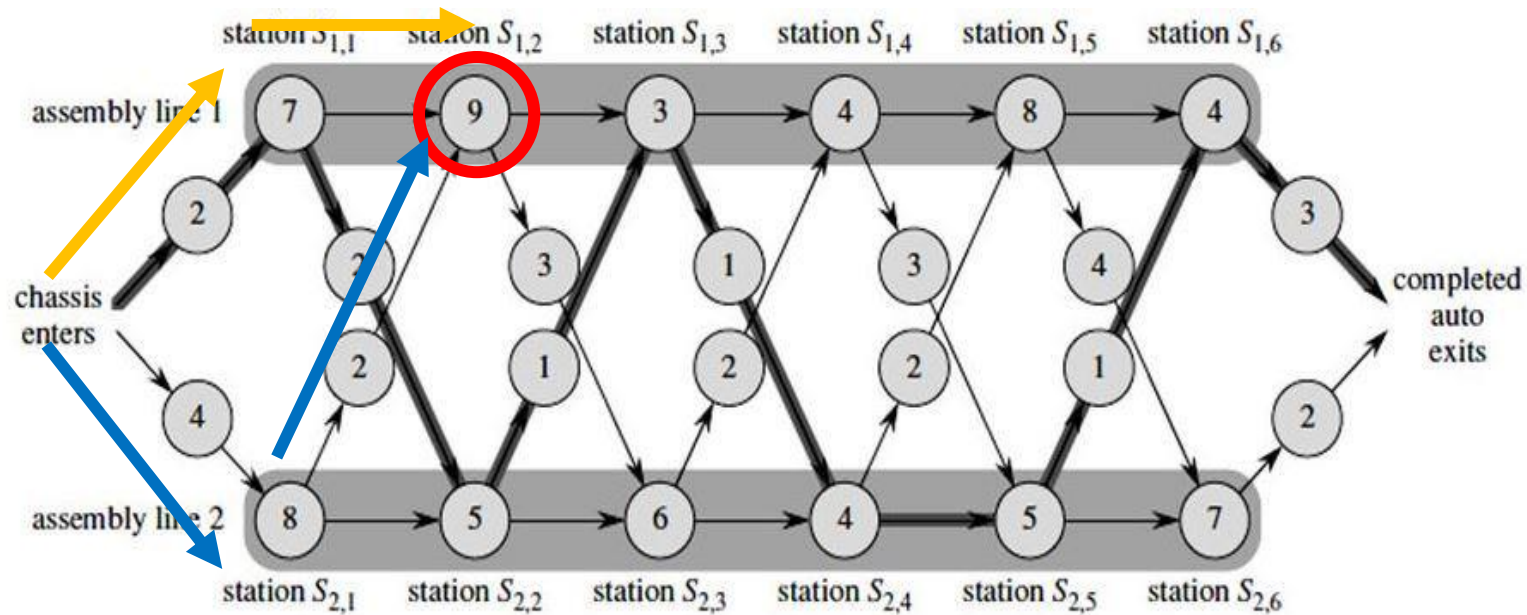
Assembly Line Scheduling – Dynamic Programming Solution



	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$
$f_1[j]$						
$f_2[j]$						

$f_1[1]$ here, $j = 1$, so, $f_1[1] = e_1 + a_{1,1}$ $f_1[1] = 2 + 7 = 9$
 $f_2[1]$ here, $j = 1$, so, $f_2[1] = e_2 + a_{2,1}$ $f_2[1] = 4 + 8 = 12$

Assembly Line Scheduling – Dynamic Programming Solution

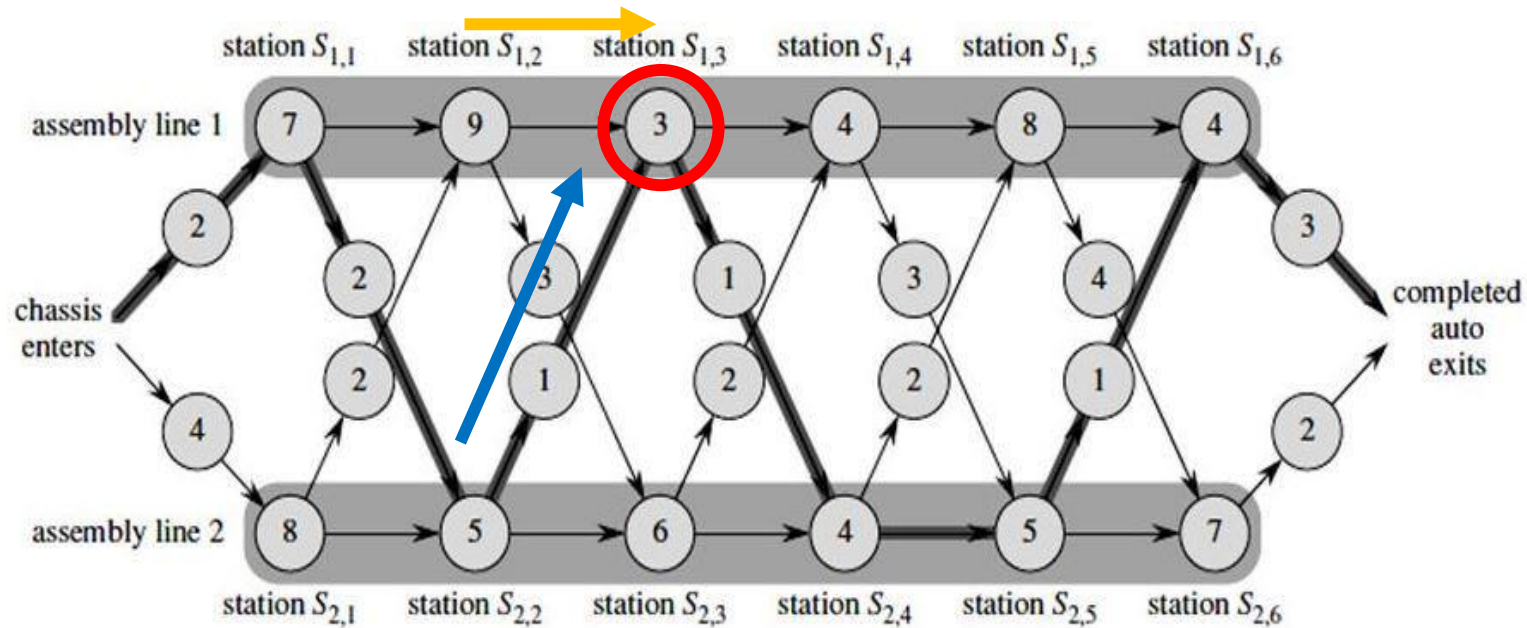


	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$
$f_1[j]$	9					
$f_2[j]$	12					

$$f_1[2] = \min(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j})$$

$$= \min(\underline{9 + 9}, \underline{12 + 2 + 9}) = \min(18, 23) = 18$$

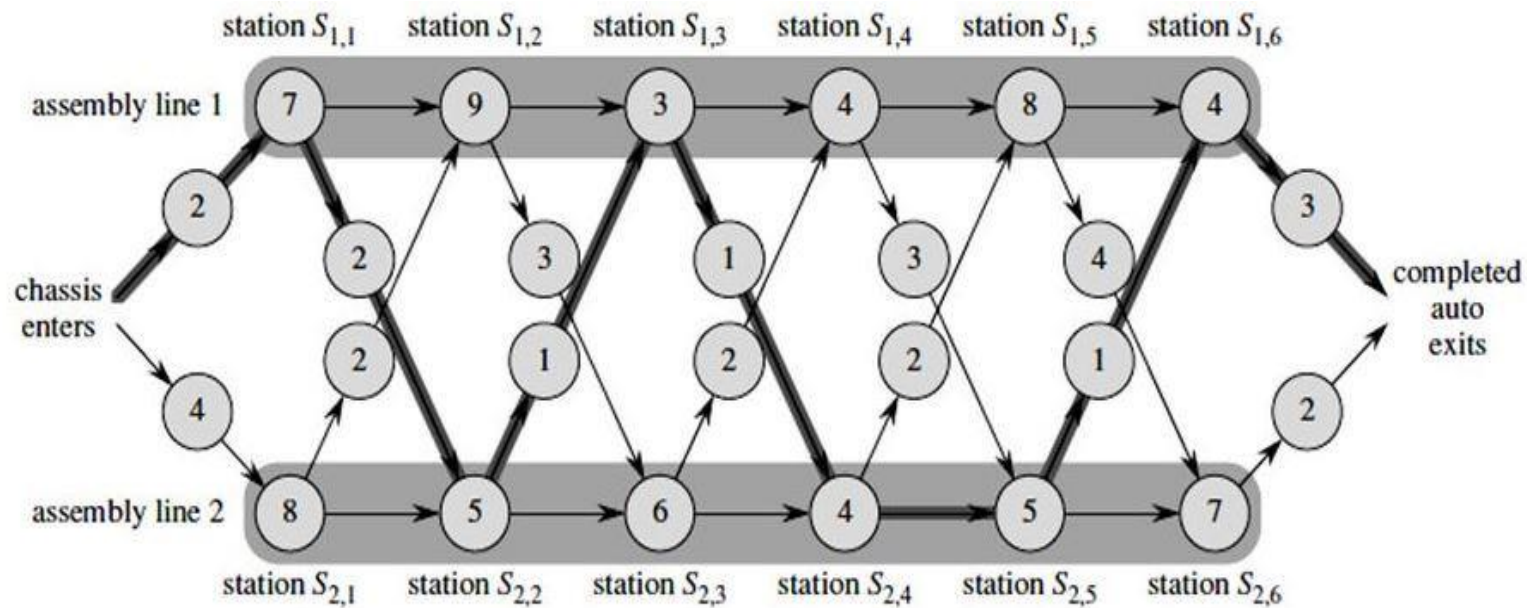
Assembly Line Scheduling – Dynamic Programming Solution



	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$
$f_1[j]$	9	18				
$f_2[j]$	12	16				

$$\text{Min} (\underline{18 + 3}, \underline{16 + 1 + 3}) = \min(21, 20) = 20$$

Assembly Line Scheduling – Dynamic Programming Solution



	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$
$f_1[j]$	9	18	20	24	32	35
$f_2[j]$	12	16	22	25	30	37

$$f^* = \min(f_1[n] + x_1, f_2[n] + x_2)$$

$$f^* = \min(35 + 3, 37 + 2) = \boxed{38}$$



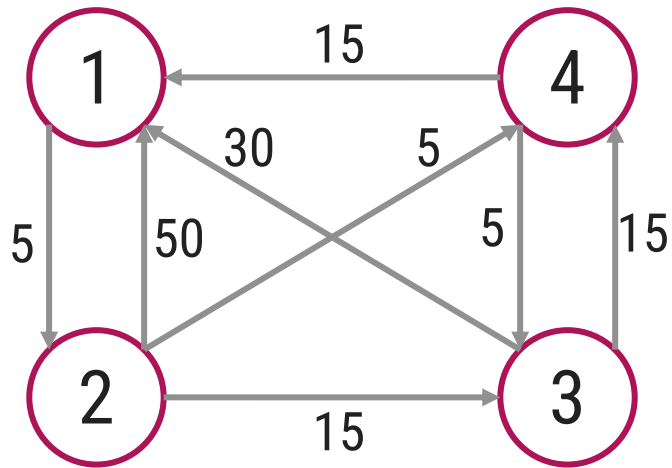
All Pair Shortest Path – Floyd's Algorithm



Introduction

- ▶ Given a **directed, connected weighted graph** $G(V, E)$, for each edge $\langle u, v \rangle \in E$, a weight $w(u, v)$ is associated with each edge.
- ▶ The all pairs of shortest paths problem is to find **a shortest path** from u to v for every pair of vertices u and v in V .
- ▶ **Floyd's algorithm** is used to find all pair shortest path problem from a given weighted graph.
- ▶ As a result of this algorithm, it will generate a matrix, which will represent the **minimum distance** from any node to all other nodes in the graph.
- ▶ At first, the output matrix is same as **given cost matrix** of the graph.
- ▶ As the algorithm proceeds, the output matrix will be updated with each vertex **k as an intermediate vertex**.
- ▶ The time complexity of this algorithm is **$O(n^3)$** , where **n** is the number of vertices in the graph.

Floyd Algorithm - Example



Step: 1 Initialization

$$D_0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \infty & 0 & 15 \\ 15 & \infty & 5 & 0 \end{pmatrix} \end{matrix}$$

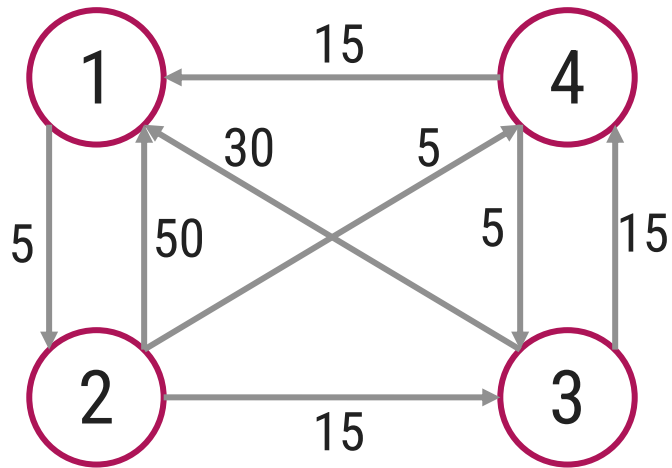
Step: 2 Calculate the distance between each node with **node 1** as an intermediate node.

$$D_1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \infty & 0 & 15 \\ 15 & \infty & 5 & 0 \end{pmatrix} \end{matrix}$$

For node 2

No change

Floyd Algorithm - Example



Step: 1 Initialization

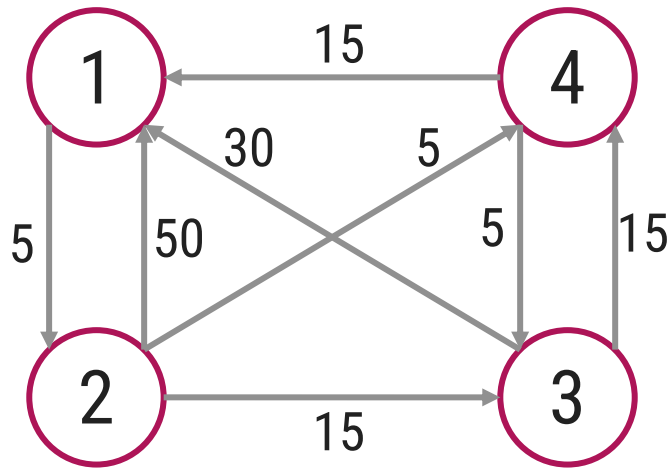
$$D_0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \infty & 0 & 15 \\ 15 & \infty & 5 & 0 \end{pmatrix} \end{matrix}$$

Step: 2 Calculate the distance between each node with **node 1** as an intermediate node.

$$D_1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \mathbf{35} & 0 & 15 \\ 15 & \infty & 5 & 0 \end{pmatrix} \end{matrix}$$

For node 3

Floyd Algorithm - Example



Step: 1 Initialization

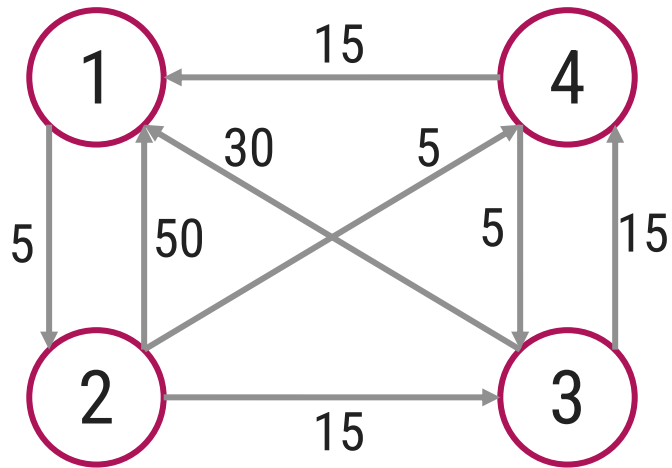
$$D_0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \infty & 0 & 15 \\ 15 & \infty & 5 & 0 \end{pmatrix} \end{matrix}$$

Step: 2 Calculate the distance between each node with **node 1** as an intermediate node.

$$D_1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \mathbf{35} & 0 & 15 \\ 15 & \mathbf{20} & 5 & 0 \end{pmatrix} \end{matrix}$$

For node 4

Floyd Algorithm - Example



Step: 1 Initialization

$$D_0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \infty & 0 & 15 \\ 15 & \infty & 5 & 0 \end{pmatrix} \end{matrix}$$

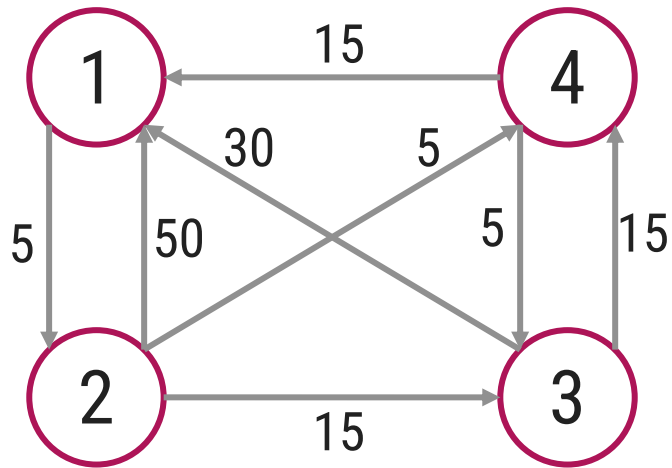
Step: 3 Calculate the distance between each node with **node 2** as an intermediate node.

$$D_2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & 20 & 10 \\ 50 & 0 & 15 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{pmatrix} \end{matrix}$$

For node 1

No change for Node 3 & 4

Floyd Algorithm - Example



Step: 1 Initialization

$$D_0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \infty & 0 & 15 \\ 15 & \infty & 5 & 0 \end{pmatrix} \end{matrix}$$

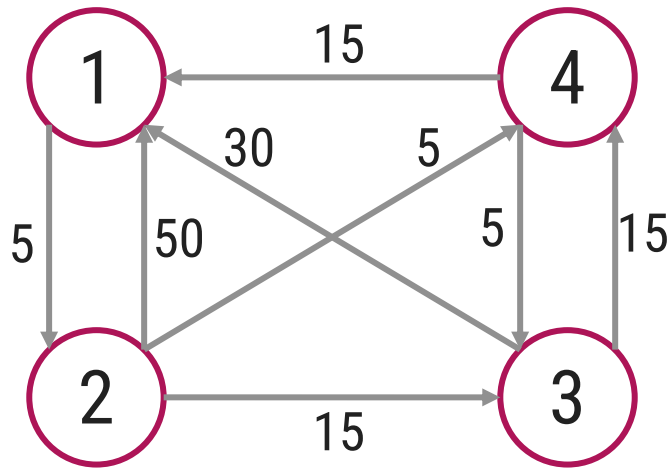
Step: 4 Calculate the distance between each node with **node 3** as an intermediate node.

$$D_3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & \mathbf{20} & \mathbf{10} \\ 50 & 0 & 15 & 5 \\ \mathbf{30} & \mathbf{35} & 0 & 15 \\ 15 & \mathbf{20} & 5 & 0 \end{pmatrix} \end{matrix}$$

For node 1

No change

Floyd Algorithm - Example



Step: 1 Initialization

$$D_0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \infty & 0 & 15 \\ 15 & \infty & 5 & 0 \end{pmatrix} \end{matrix}$$

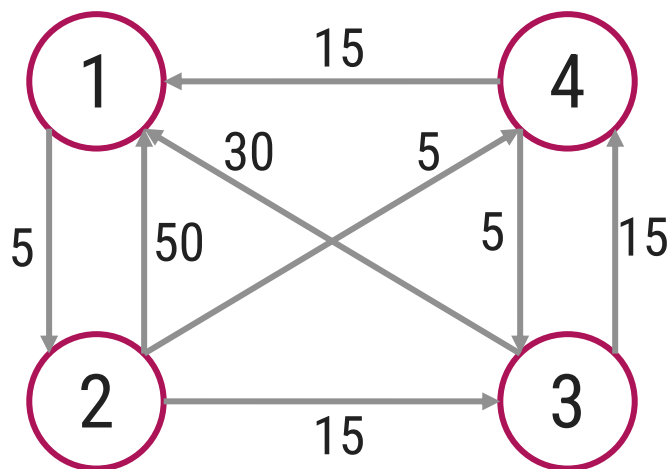
Step: 4 Calculate the distance between each node with **node 3** as an intermediate node.

$$D_3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & \mathbf{20} & \mathbf{10} \\ \mathbf{45} & 0 & 15 & 5 \\ \mathbf{30} & \mathbf{35} & 0 & 15 \\ 15 & \mathbf{20} & 5 & 0 \end{pmatrix} \end{matrix}$$

For node 2

No change for Node 4

Floyd Algorithm - Example



Step: 1 Initialization

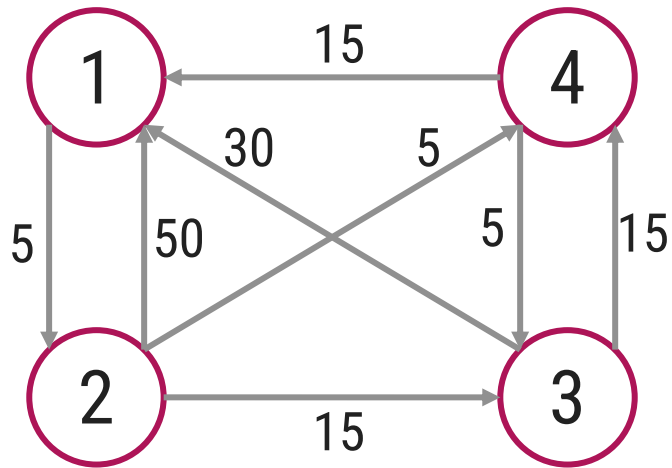
$$D_0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \infty & 0 & 15 \\ 15 & \infty & 5 & 0 \end{pmatrix} \end{matrix}$$

Step: 5 Calculate the distance between each node with **node 4** as an intermediate node.

$$D_4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & 15 & 10 \\ 45 & 0 & 15 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{pmatrix} \end{matrix}$$

For node 1	

Floyd Algorithm - Example



Step: 1 Initialization

$$D_0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \infty & 0 & 15 \\ 15 & \infty & 5 & 0 \end{pmatrix} \end{matrix}$$

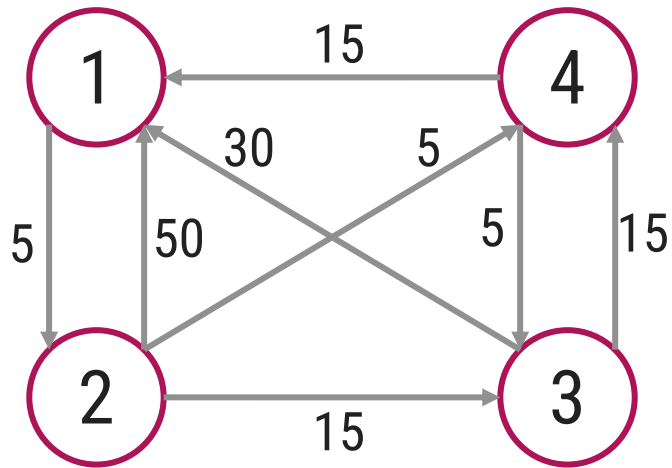
Step: 5 Calculate the distance between each node with **node 4** as an intermediate node.

$$D_4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & 15 & 10 \\ 20 & 0 & 10 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{pmatrix} \end{matrix}$$

For node 2

No change for Node 3

Floyd Algorithm - Example



Step: 1 Initialization

$$D_0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \infty & 0 & 15 \\ 15 & \infty & 5 & 0 \end{pmatrix} \end{matrix}$$

Step: 5 Calculate the distance between each node with **node 4** as an intermediate node.

$$D_4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 5 & \mathbf{15} & \mathbf{10} \\ \mathbf{20} & 0 & \mathbf{10} & 5 \\ 30 & \mathbf{35} & 0 & 15 \\ 15 & \mathbf{20} & 5 & 0 \end{pmatrix} \end{matrix}$$

Final Solution

Floyd's Algorithm

```
function Floyd( $L[1..n, 1..n]$ ):array  $[1..n, 1..n]$   
    array  $D[1..n, 1..n]$   
     $D \leftarrow L$   
    for  $k \leftarrow 1$  to  $n$  do  
        for  $i \leftarrow 1$  to  $n$  do  
            for  $j \leftarrow 1$  to  $n$  do  
                 $D[i,j] \leftarrow \min(D[i,j], D[i,k] + D[k,j])$   
    return  $D$ 
```



Chain Matrix Multiplication



Introduction

- ▶ Let matrix $A (p \times q)$ and matrix $B (q \times r)$ are two given matrices
- ▶ The resultant matrix $C = A \cdot B$
- ▶ So,

$$C_{ij} = \sum_{k=1}^q a_{ik} b_{kj} \quad \text{Where, } 1 \leq i \leq p, 1 \leq j \leq r$$

- ▶ Suppose, matrix $A (2 \times 3)$ and matrix $B (3 \times 2)$ are given
- ▶ The product $C = A \cdot B$ and the dimension of matrix C is 2×2
- ▶ The total number of scalar multiplications required will be,

$$p * q * r = 2 * 3 * 2 = 12$$

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{bmatrix} \text{ and } B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \\ B_{31} & B_{32} \end{bmatrix}$$

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21} + A_{13} \cdot B_{31}$$

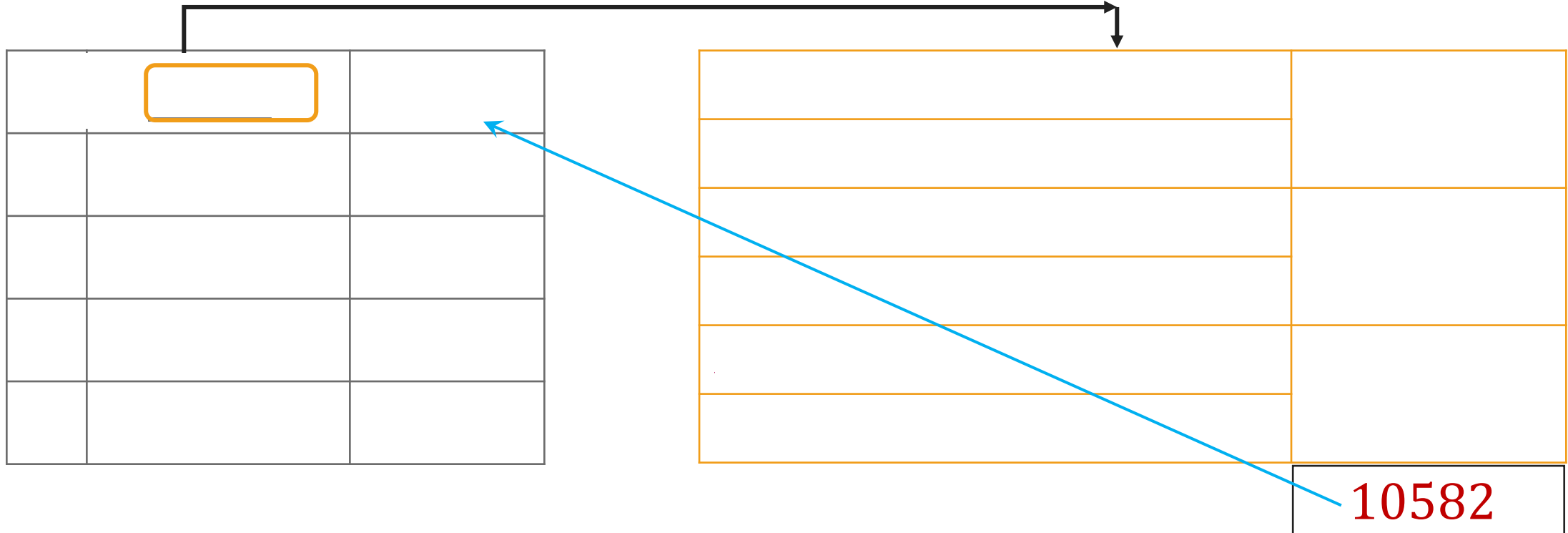
$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22} + A_{13} \cdot B_{32}$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21} + A_{23} \cdot B_{31}$$

$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22} + A_{23} \cdot B_{32}$$

Matrix Chain Multiplication

- ▶ Now, we want to calculate **the product of more than two matrices**. Matrix multiplication is **associative**.
- ▶ The product of **four matrices** $A(13 \times 5)$, $B(5 \times 89)$, $C(89 \times 3)$, $D(3 \times 34)$ can be fully parenthesized in **5 distinct ways**.



Matrix Chain Multiplication

- ▶ Now, we want to calculate **the product of more than two matrices**. Matrix multiplication is **associative**.
- ▶ The product of **four matrices** $A(13 \times 5)$, $B(5 \times 89)$, $C(89 \times 3)$, $D(3 \times 34)$ can be fully parenthesized in **5 distinct ways**.

1	$((AB)C)D$	10582
		54201

Matrix Chain Multiplication using Dynamic Programming

- ▶ Matrix table $M[i][j]$ stores the **cost of multiplications**.

To generate $M[i][j]$ use following steps:

Step-1: if $i = j$ then $M[i][j] = 0$

Step-2: if $j = i + 1$ then $M[i][j] = P_{i-1} \cdot P_i \cdot P_{i+1}$

Step-3: if $i < j$ then

$$M[i][j] = \min(M[i][k] + M[k + 1][j] + P_{i-1} \cdot P_k \cdot P_j) \text{ with } i \leq k < j$$

Matrix Chain Multiplication using Dynamic Programming

- ▶ Here dimensions are, $P_0 = 5, P_1 = 4, P_2 = 6, P_3 = 2, P_4 = 7$.
- ▶ There would be 4 matrices $A(5 \times 4), B(4 \times 6), C(6 \times 2)$ and $D(2 \times 7)$.

Step-2: if $M[i][j] > P_i \cdot P_k \cdot P_{i+1}$ then $M[i][j] = P_i \cdot P_k \cdot P_{i+1}$

So, $M[1][2] = P_0 \cdot P_1 \cdot P_2 = 5 \cdot 4 \cdot 6 = 120$

	1	2	3	4
1				
2	--			
3	--	--		
4	--	--	--	

Matrix Chain Multiplication using Dynamic Programming

► Here dimensions are,

$$\rightarrow P_0 = 5, P_1 = 4, P_2 = 6, P_3 = 2, P_4 = 7.$$

step 3:

$M[i][j]$

$$= \min(M[i][k] + M[k+1][j] + P_{i-1} \cdot P_k \cdot P_j) \\ \text{with } i \leq k < j$$

	1	2	3	4
1	0	120		
2	--	0	48	
3	--	--	0	84
4	--	--	--	0

$M[1][3]$, here $i < j$

for $i = 1, j = 3$ and $k = 1$ or 2

because $i \leq k \leq j - 1$

For $k = 1$

$M[1][3]$

$$= M[1][1] + M[2][3] + P_0 * P_1 * P_3 \\ = 0 + 48 + 5 * 4 * 2$$

$$M[1][3] = \mathbf{88}$$

For $k = 2$

$M[1][3]$

$$= M[1][2] + M[3][3] + P_0 * P_2 * P_3 \\ = 120 + 0 + 5 * 6 * 2$$

$$M[1][3] = \mathbf{180}$$

Matrix Chain Multiplication using Dynamic Programming

► Here dimensions are,

$$\rightarrow P_0 = 5, P_1 = 4, P_2 = 6, P_3 = 2, P_4 = 7.$$

step 3:

$M[i][j]$

$$= \min(M[i][k] + M[k+1][j] + P_{i-1} \cdot P_k \cdot P_j) \\ \text{with } i \leq k < j$$

	1	2	3	4
1	0	120	88	
2	--	0	48	
3	--	--	0	84
4	--	--	--	0

$M[2][4]$, here $i < j$

for $i = 2, j = 4$ and $k = 2$ or 3

because $i \leq k \leq j - 1$

For $k = 2$

$M[2][4]$

$$= M[2][2] + M[3][4] + P_1 * P_2 * P_4 \\ = 0 + 84 + 4 * 6 * 7$$

$$M[1][3] = 252$$

For $k = 3$

$M[2][4]$

$$= M[2][3] + M[4][4] + P_1 * P_3 * P_4 \\ = 48 + 0 + 4 * 2 * 7$$

$$M[2][4] = \mathbf{104}$$

Matrix Chain Multiplication using Dynamic Programming

► Here dimensions are,

$$\rightarrow P_0 = 5, P_1 = 4, P_2 = 6, P_3 = 2, P_4 = 7.$$

step 3:

$M[i][j]$

$$= \min(M[i][k] + M[k+1][j] + P_{i-1} \cdot P_k \cdot P_j) \\ \text{with } i \leq k < j$$

	1	2	3	4
1	0	120	88	
2	--	0	48	104
3	--	--	0	84
4	--	--	--	0

$M[1][4]$, here $i < j$

for $i = 1, j = 4$ and $k = 1$ or 2 or 3

because $i \leq k \leq j - 1$

For $k = 1$

$M[1][4]$

$$= M[1][1] + M[2][4] + P_0 * P_1 * P_4 \\ = 0 + 104 + 5 * 4 * 7 = \mathbf{244}$$

For $k = 2$

$M[1][4]$

$$= M[1][2] + M[3][4] + P_0 * P_2 * P_4 \\ = 120 + 84 + 5 * 6 * 7 = \mathbf{414}$$

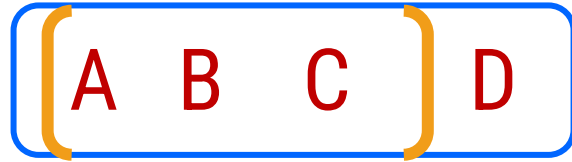
For $k = 3$

$M[1][4]$

$$= M[1][3] + M[4][4] + P_0 * P_3 * P_4 \\ = 88 + 0 + 5 * 2 * 7 = \mathbf{158}$$

Matrix Chain Multiplication using Dynamic Programming

► How to parenthesize matrices?



	1	2	3	4
1	0	120	88	158
2	--	0	48	104
3	--	--	0	84
4	--	--	--	0

$M[1][4]$, here $i < j$

for $i = 1, j = 4$ and $k = 1$ or 2 or 3

because $i \leq k \leq j - 1$

For $k = 1$

$M[1][4]$

$$\begin{aligned} &= M[1][1] + M[2][4] + P_0 * P_1 * P_4 \\ &= 0 + 104 + 5 * 4 * 7 = \mathbf{244} \end{aligned}$$

For $k = 2$

$M[1][4]$

$$\begin{aligned} &= M[1][2] + M[3][4] + P_0 * P_2 * P_4 \\ &= 120 + 84 + 5 * 6 * 7 = \mathbf{414} \end{aligned}$$

For $k = 3$

$M[1][4]$

$$\begin{aligned} &= M[1][3] + M[4][4] + P_0 * P_3 * P_4 \\ &= 88 + 0 + 5 * 2 * 7 = \mathbf{158} \end{aligned}$$

Matrix Chain Multiplication using Dynamic Programming

► How to parenthesize matrices?



	1	2	3	4
1	0	120	88	158
2	--	0	48	104
3	--	--	0	84
4	--	--	--	0

$M[1][3]$, here $i < j$

for $i = 1, j = 3$ and $k = 1$ or 2

because $i \leq k \leq j - 1$

For $k = 1$

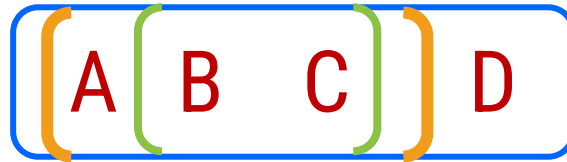
$$\begin{aligned} M[1][3] &= M[1][1] + M[2][3] + P_0 * P_1 * P_3 \\ &= 0 + 48 + 5 * 4 * 2 \\ M[1][3] &= \mathbf{88} \end{aligned}$$

For $k = 2$

$$\begin{aligned} M[1][3] &= M[1][2] + M[3][3] + P_0 * P_2 * P_3 \\ &= 120 + 0 + 5 * 6 * 2 \\ M[1][3] &= \mathbf{180} \end{aligned}$$

Matrix Chain Multiplication using Dynamic Programming

- ▶ Here dimensions are, $P_0 = 5, P_1 = 4, P_2 = 6, P_3 = 2, P_4 = 7$.
- ▶ There would be 4 matrices $A(5 \times 4), B(4 \times 6), C(6 \times 2)$ and $D(2 \times 7)$.



$R1 = B.C$	$4 * 6 * 2 = 48$
$R2 = A.R1$	$5 * 4 * 2 = 40$
$R3 = R1.D$	$5 * 2 * 7 = 70$

158



Longest Common Subsequence



Introduction

- ▶ A subsequence is a sequence that appears in the same relative order, **but not necessarily contiguous**.
- ▶ Given two sequences X and Y , we say that a sequence Z is a common subsequence of X and Y if Z is a subsequence of both X and Y .
- ▶ E.g., if $X = \langle A, \underline{B}, \underline{C}, B, D, \underline{A}, B \rangle$ and $Y = \langle \underline{B}, D, \underline{C}, A, B, \underline{A} \rangle$ then $\langle B, C, A \rangle$ is a subsequence.
- ▶ Use dynamic programming technique to find **the longest common subsequence (LCS)**.

LCS – Optimal Sub-structure

► We need to generate table $c(1..m, 1..n)$

where m = length of string $S1$ and n = length of string $S2$.

To generate table $c[i][j]$ use following steps:

Step-1: Make $c[i][0] = 0$ and $c[0][j] = 0$

Step-2: if $x_i = y_j$ then $c[i, j] \leftarrow c[i - 1, j - 1] + 1$

Step-3: else $c[i, j] \leftarrow \max(c[i - 1, j], c[i, j - 1])$

LCS – Dynamic Programming Solution

if $x_i = y_j$ *then* $c[i, j] \leftarrow c[i - 1, j - 1] + 1$ *else* $c[i, j] \leftarrow \max(c[i - 1, j], c[i, j - 1])$

	y_j	A	<i>B</i>	<i>C</i>	<i>B</i>	<i>D</i>	<i>A</i>	<i>B</i>
x_i								
B								
<i>D</i>								
<i>C</i>								
<i>A</i>								
<i>B</i>								
<i>A</i>								

LCS – Dynamic Programming Solution

if $x_i = y_j$ *then* $c[i, j] \leftarrow c[i - 1, j - 1] + 1$ *else* $c[i, j] \leftarrow \max(c[i - 1, j], c[i, j - 1])$

	y_j	A	<i>B</i>	<i>C</i>	<i>B</i>	<i>D</i>	<i>A</i>	<i>B</i>
x_i	0	0	0	0	0	0	0	0
B	0	↑ (0)	↖ 1	← 1	↖ 1	← 1	← 1	↖ 1
<i>D</i>	(0)	↑	↑	↑	↑	↖	←	←
<i>C</i>	0							
<i>A</i>	0							
<i>B</i>	0							
<i>A</i>	0							

LCS – Dynamic Programming Solution

if $x_i = y_j$ *then* $c[i, j] \leftarrow c[i - 1, j - 1] + 1$ *else* $c[i, j] \leftarrow \max(c[i - 1, j], c[i, j - 1])$

	y_j	A	<i>B</i>	<i>C</i>	<i>B</i>	<i>D</i>	<i>A</i>	<i>B</i>
x_i	0	0	0	0	0	0	0	0
<i>B</i>	0	↑ 0	↖ 1	← 1	↖ 1	← 1	← 1	↖ 1
D	0	↑ 0	↑ 1	↑ 1	↑ 1	↖ 2	← 2	← 2
<i>C</i>	0	↑	↑	↖	←	↑	↑	↑
<i>A</i>	0	↖	↑	↑	↑	↑	↖	←
<i>B</i>	0	↑	↖	↑	↖	←	↑	↖
<i>A</i>	0	↖	↑	↑	↑	↑	↖	↑

LCS – Dynamic Programming Solution

$X = \langle A, B, C, B, D, A, B \rangle$ and $Y = \langle B, D, C, A, B, A \rangle$ and $LCS = \langle B, D, A, B \rangle$

	y_j	<i>A</i>	<i>B</i>	<i>C</i>	<i>B</i>	<i>D</i>	<i>A</i>	<i>B</i>
x_i	0	0	0	0	0	0	0	0
<i>B</i>	0	↑ 0	↖ 1	← 1	↖ 1	← 1	← 1	↖ 1
<i>D</i>	0	↑ 0	↑ 1	↑ 1	↑ 1	↖ 2	← 2	← 2
<i>C</i>	0	↑ 0	↑ 1	↖ 2	← 2	↑ 2	↑ 2	↑ 2
<i>A</i>	0	↖ 1	↑ 1	↑ 2	↑ 2	↑ 2	↖ 3	← 3
<i>B</i>	0	↑ 1	↖ 2	↑ 2	↖ 3	← 3	↑ 3	↖ 4
<i>A</i>	0	↖ 1	↑ 2	↑ 2	↑ 3	↑ 3	↖ 4	↑ 4



Thank You!



SUBSCRIBE

